LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 5
Student Name: S Satya Jayavanth
Roll No.: A24126552116
Date: 29-08-2026

---

1. TITLE

Implement Modules in NodeJS

---

2. AIM

To implement and demonstrate the use of **modules in NodeJS** using CommonJS modules, ES6 modules, and built-in NodeJS modules.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in NodeJS.
- To create and export functions using a CommonJS module.
- To import and use functions from another JavaScript file using require().
- To understand ES6 module syntax using export and import.
- To create reusable functions and access them from different files.
- To use the built-in os module of NodeJS.
- To retrieve operating system information using NodeJS.
- To understand the use of the path built-in module.

---

4. THEORY

NodeJS supports a **module system** that allows a program to be divided into multiple files. Modules help organize code into smaller, reusable and maintainable components. A module can contain functions, variables or other code that can be exported and used by another file.

NodeJS supports **CommonJS modules**, where module.exports is used to export functionality and require() is used to import it. In this experiment, arithmetic functions such as addition, subtraction, multiplication, division and remainder are exported from one file and accessed from another file.
NodeJS also supports **ES6 modules**. The export keyword is used to make a function available outside a module, while the import keyword is used to access the exported function in another file. The experiment demonstrates this approach using a square() function.
NodeJS also provides several **built-in modules** that provide useful functionality without requiring external installation. The os module provides information about the operating system, memory, user information, platform and CPU architecture. The path module provides utilities for working with file and directory paths.

---

5. ALGORITHM / PROCEDURE

A. CommonJS Module

1. Create a JavaScript file named file1.js.
2. Define functions for addition, subtraction, multiplication, division and remainder.
3. Export the functions using module.exports.
4. Create another JavaScript file named file2.js.
5. Import file1.js using the require() function.
6. Call the exported arithmetic functions.
7. Display the results using console.log().

B. ES6 Module

1. Create a JavaScript file named file3.js.
2. Define a square() arrow function.
3. Export the function using the export keyword.
4. Create another file named file4.js.
5. Import the square() function using the import statement.
6. Pass a number to the function.
7. Display the square of the number.

C. Built-in NodeJS Module

1. Import the os module using the NodeJS module syntax.
2. Use os.type() to obtain the operating system type.
3. Use os.totalmem() to obtain the total system memory.
4. Use os.freemem() to obtain the available memory.
5. Use os.userInfo() to obtain current user information.
6. Use os.platform() to obtain the operating system platform.
7. Use os.arch() to obtain the CPU architecture.
8. Import the path module.
9. Execute the program using NodeJS and observe the output.

---

6. PROGRAM

File1.js

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const mul = (a,b)=> a*b ;
const div=(a,b)=> a/b;
const rem=(a,b)=>a%b;
module.exports = { add, subtract ,mul,div,rem};
```

File2.js

```
const cal = require('./FS_Prog82.js');
const fs = require('fs');

console.log("add of 5,3 is:",cal.add(5, 3));
console.log("mul of 5,3 is:",cal.mul(5, 3));
console.log("sub of 5,3 is:",cal.subtract(5, 3));
console.log("div of 5,3 is:",cal.div(6, 3));
console.log("rem of 5,3 is:",cal.rem(8, 3));
```

File3.js

```
export const square=(a)=> a*a;
```

File4.js

```
import { square } from './FS_Prog84.js';
console.log(square(5));
```

File5.js

```
import os from 'node:os';

console.log('Operating System:', os.type());
console.log('Total Memory (GB):', os.totalmem() / 1024 / 1024 / 1024);
console.log('Free Memory (GB):', os.freemem() / 1024 / 1024 / 1024);
console.log(`Current User Info:`, os.userInfo());
console.log(`OS Platform: ${os.platform()}`);
console.log(`CPU Architecture: ${os.arch()}`);

import path from 'node:path';
```

---

7. CODE EXPLANATION

File 1 – CommonJS Module

const add = (a, b) => a + b;

Defines an arrow function to calculate the sum of two numbers.

subtract()

Calculates the difference between two numbers.

mul()

Calculates the product of two numbers.

div()

Calculates the division of two numbers.

rem()

Calculates the remainder after division.

module.exports = { add, subtract, mul, div, rem };

Exports the functions so that they can be accessed from another JavaScript file.

File 2 – Importing CommonJS Module

const cal = require('./file1.js');

Imports the functions exported from file1.js.

cal.add(5, 3)

Calls the add() function using the imported module.

cal.mul(5, 3)

Calls the multiplication function.

cal.subtract(5, 3)

Calls the subtraction function.

cal.div(6, 3)

Calls the division function.

cal.rem(8, 3)

Calls the remainder function.

console.log()

Displays the calculated results in the NodeJS console.

File 3 – ES6 Module

export const square = (a) => a * a;

Defines an arrow function to calculate the square of a number and exports it using the ES6 export keyword.

File 4 – Importing ES6 Module

import { square } from './file3.js';

Imports the square() function from file3.js.

square(5)

Calls the imported function with the value 5.

File 5 – OS Module

import os from 'node:os';

Imports NodeJS's built-in os module.

os.type()

Returns the operating system name.

os.totalmem()

Returns the total amount of system memory in bytes.

os.freemem()
Returns the amount of currently available system memory in bytes.
os.userInfo()
Returns information about the current operating system user.
os.platform()
Returns the operating system platform.
os.arch()
Returns the CPU architecture.
import path from 'node:path';
Imports NodeJS's built-in path module for working with file and directory paths.

---

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

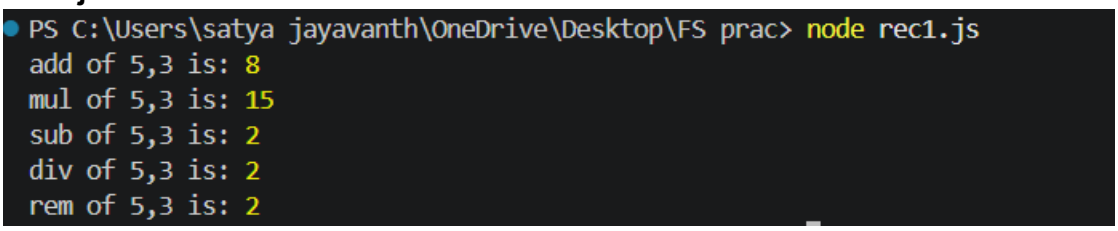
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Import file1.js using require()	Module should be imported successfully	Module imported successfully	Pass
TC-02	Call add(5, 3)	Result should be 8	8 displayed	Pass
TC-03	Call mul(5, 3)	Result should be 15	15 displayed	Pass
TC-04	Call subtract(5, 3)	Result should be 2	2 displayed	Pass
TC-05	Call div(6, 3)	Result should be 2	2 displayed	Pass
TC-06	Call rem(8, 3)	Result should be 2	2 displayed	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-07	Import square() using ES6 import	Function should be imported successfully	Function imported successfully	Pass
TC-08	Call square(5)	Result should be 25	25 displayed	Pass
TC-09	Execute the OS module program	Operating system information should be displayed	OS information displayed	Pass
TC-10	Execute os.totalmem() and os.freemem()	Memory information should be displayed	Memory information displayed	Pass
TC-11	Execute os.userInfo()	Current user information should be displayed	User information displayed	Pass
TC-12	Execute os.platform() and os.arch()	Platform and CPU architecture should be displayed	Information displayed	Pass

---

9. OUTPUT

File2.js



File4.js

```
PS C:\Users\satya jayavanth\OneDrive\Desktop\FS prac> node rec1.js
25
```

File5.js

```
PS C:\Users\satya jayavanth\OneDrive\Desktop\FS prac> node rec1.js
Operating System: Windows_NT
Total Memory (GB): 15.700065612792969
Free Memory (GB): 4.42462158203125
Current User Info: [Object: null prototype] {
  uid: -1,
  gid: -1,
  username: 'satya jayavanth',
  homedir: 'C:\\Users\\satya jayavanth',
  shell: null
}
OS Platform: win32
CPU Architecture: x64
```

---

10. RESULT

Thus, the **NodeJS module system** was successfully implemented and demonstrated. CommonJS modules were created using module.exports and imported using require(). ES6 modules were implemented using export and import. The built-in os module was also used successfully to obtain operating system, memory, user, platform and CPU architecture information.